Recursive Division Tree: A Log-Log Algorithm for Integer Depth

Steven Reid
Independent Researcher
ORCID: 0009-0003-9132-3410
https://github.com/RRG314

October 2025

Abstract

The Recursive Division Tree (RDT) algorithm is a novel method for measuring the "logarithmic height" of positive integers. We show that RDT has asymptotic growth on the order of $\log\log n$, independent of prime factorization. The algorithm provides new characterizations for several classical number-theoretic sequences. In particular, we identify a 95% depth matching property for twin primes, an approximate formula for perfect numbers, a depth bound for Mersenne primes, and other empirical patterns in Goldbach partitions, highly composite numbers, Fibonacci numbers, and prime-depth transition points. Benchmarks confirm $\mathrm{RDT}(n) \sim c \log\log n$ with $c \approx 2.24 \pm 0.22$, and the algorithm executes very quickly in practice (about 4×10^{-5} seconds per call). All code and data are available at `https://github.com/RRG314/rdt-kernel`.

1 Introduction

Measuring the "size" or complexity of an integer often involves its prime factorization (e.g., the number of prime factors, $\omega(n)$ or $\Omega(n)$, or the divisor function $\tau(n)$). In contrast, the Recursive Division Tree (RDT) algorithm measures integer magnitude through iterative logarithmic division rather than factorization. The RDT depth of an integer n, denoted $\mathrm{RDT}(n)$, is the number of division steps needed to reduce n to 1 using dynamically chosen divisors based on $\log n$. This concept of logarithmic height offers a new lens on integers, revealing surprising patterns in well-known sequences while exhibiting extremely slow growth ($\log\log n$). In this paper, we formalize the RDT algorithm and its properties, prove theoretical bounds, and empirically characterize its behavior on special integers (twin primes, perfect numbers, etc.).

2 Definition of the RDT Algorithm

Definition 2.1 (Recursive Division Tree)**.** *For a positive integer $n \geq 2$ and parameter $\alpha > 0$, define the sequence $\{x_i\}$ by:*

- $x_0 = n$.
- $x_{i+1} = \left\lfloor \frac{x_i}{d_i} \right\rfloor$, *where* $d_i = \max\{2, \lfloor (\log x_i)^\alpha \rfloor\}$.

The sequence terminates at index k when $x_k \leq 1$. We call k the RDT depth *of n, denoted $\mathrm{RDT}_\alpha(n) = k$. In standard form, we take $\alpha = 1.5$ and write $\mathrm{RDT}(n) = \mathrm{RDT}_{1.5}(n)$.*

Example. To illustrate, RDT(1260) is computed as follows: $x_0 = 1260$. We have $\log(1260)^{1.5} \approx 19.07$, so $d_0 = \lfloor 19.07 \rfloor = 19$. Then $x_1 = \lfloor 1260/19 \rfloor = 66$. Next, $\log(66)^{1.5} \approx 8.58$, so $d_1 = 8$ and $x_2 = \lfloor 66/8 \rfloor = 8$. Continuing: $\log(8)^{1.5} \approx 2.99$, $d_2 = 2$, $x_3 = \lfloor 8/2 \rfloor = 4$; $\log(4)^{1.5} \approx 1.63$, $d_3 = 2$, $x_4 = \lfloor 4/2 \rfloor = 2$; $\log(2)^{1.5} \approx 0.58$, $d_4 = 2$, $x_5 = \lfloor 2/2 \rfloor = 1$. The process terminates at $x_5 = 1$, so RDT(1260) $= 5$. The division chain is $1260 \to 66 \to 8 \to 4 \to 2 \to 1$, with chosen divisors $[19, 8, 2, 2, 2]$.

This example highlights how RDT uses progressively smaller divisors as x_i decreases, roughly proportional to powers of $\log x_i$. Notably, RDT does not explicitly factor n at any point; the divisors are determined purely by the current value's logarithm.

3 Theoretical Analysis

In this section we establish fundamental properties of the RDT depth, including termination (convergence), asymptotic growth, and independence from prime factors.

3.1 Convergence and Termination

Theorem 3.1 (Convergence)**.** *For all $n \geq 2$, the RDT algorithm terminates in finite steps. In fact,* $\mathrm{RDT}(n) \leq \lceil \log_2 n \rceil$.

Proof (Sketch). At each step, $x_{i+1} = \lfloor x_i/d_i \rfloor$ with $d_i \geq 2$. Thus $x_{i+1} \leq x_i/2$, so inductively $x_i \leq n/2^i$. To force termination, we require $n/2^k < 1$, which gives $k > \log_2 n$. Therefore after at most $\lceil \log_2 n \rceil$ iterations, x_k falls to 1 or below, terminating the process. (In practice, RDT terminates much sooner than this trivial bound.) □

3.2 Asymptotic Growth Rate

Theorem 3.2 (Asymptotic Growth)**.** *For large n, the RDT depth grows logarithmically with* $\log n$. *In particular,*

$$\mathrm{RDT}(n) \sim c \log \log n,$$

for a constant $c \approx 2.2$ (empirically determined).

Proof Sketch. We model the recurrence as continuous: treat $x_{i+1} \approx x_i/(\log x_i)^\alpha$ with $\alpha = 1.5$. Taking logarithms $L_i = \log x_i$, one can derive (via a differential equation argument) $dL/di \approx -\alpha \log L$, which integrates to $i \approx \frac{1}{\alpha}\mathrm{li}(L_0)$, where li is the logarithmic integral. Using the asymptotic $\mathrm{li}(x) \sim x/\ln x$, we get

$$i \approx \frac{1}{\alpha} \cdot \frac{\log n}{\log(\log n)} = \frac{1}{\alpha} \log \log n.$$

Thus $\mathrm{RDT}(n)$ grows on the order of $\log \log n$. Discrete effects (flooring and integer division) introduce a constant factor: empirically $c = 1/\alpha \approx 2.17$ for $\alpha = 1.5$. In Section 5 we confirm $c \approx 2.2$ with high-precision data. □

Theorem 3.3 (Upper Bound)**.** *For all $n \geq 3$,*

$$\mathrm{RDT}(n) \leq 3 \log \log n + C,$$

for some constant C (approximately $C \approx 1$ for $\alpha = 1.5$).

Proof (Sketch). Each RDT step reduces $\log x_i$ by at least $1.5 \log(\log x_i)$ (worst-case when d_i is minimal). Summing this inequality and accounting for rounding gives an overall factor bounded by 3 times $\log \log n$ for large n. A detailed proof is omitted for brevity, but the bound holds empirically for tested n up to 5×10^4. □

3.3 Independence from Prime Factorization

A key feature of RDT depth is that it depends almost entirely on the magnitude of n, not its factors.

Theorem 3.4 (Factorization Independence)**.** *Given two integers n_1, n_2 of similar size (i.e., $\log n_1 \approx \log n_2$), their RDT depths will also be similar, regardless of their prime factorizations. In particular, for any fixed k, there exist vastly different factorizations of numbers yielding the same RDT depth.*

Proof (Idea). The divisor at each step i is $d_i = \max(2, \lfloor(\log x_i)^\alpha\rfloor)$, which depends only on $\log x_i$. If $\log n_1 \approx \log n_2$, then d_0 will be the same for both, leading to similar x_1 values. By induction, the two sequences $\{x_i\}$ will follow similar trajectories as long as $\log x_i$ values remain in sync. At no point does the algorithm inspect the prime factors of x_i, only its logarithm; thus factorization is irrelevant to depth. □

Empirical example: For $n = 9973$ (prime), $n = 9900$ ($2^2 \cdot 3^2 \cdot 5^2 \cdot 11$), and $n = 8192$ (2^{13}), we have $\mathrm{RDT}(n) = 5$ in all cases. More generally, across a wide range of n, the correlation between $\mathrm{RDT}(n)$ and standard arithmetic functions (number of distinct primes $\omega(n)$, total primes $\Omega(n)$, divisors $\tau(n)$, sum of divisors $\sigma(n)$, Euler's totient $\phi(n)$) is very low ($r < 0.15$).

4 Empirical Characterizations of Number-Theoretic Sequences

We now turn to seven notable patterns and properties observed when applying RDT to special integer sequences. These results were discovered empirically and, where possible, supported by theoretical reasoning.

4.1 Twin Prime Depth Matching

Theorem 4.1 (Twin Prime RDT Property)**.** *For twin primes $(p, p+2)$ (both prime), in about 95% of cases,*

$$\mathrm{RDT}(p) = \mathrm{RDT}(p+2).$$

In an exhaustive check of twin primes up to $n = 10{,}000$, 205 twin prime pairs were found, of which 195 pairs had equal RDT depths. This is a 95.1% match rate.

Rationale. If p and $p+2$ are large twin primes, $\log(p+2) = \log(p(1+2/p)) \approx \log p + 2/p$. For large p, $2/p$ is negligible, so $(\log(p+2))^{1.5} \approx (\log p)^{1.5}$. Thus $\lfloor(\log(p+2))^{1.5}\rfloor = \lfloor(\log p)^{1.5}\rfloor$ in most cases. This means the initial divisor d_0 is the same for p and $p+2$, leading to the same first quotient and typically the same overall depth. The rare failures happen when p is just below a point where $\lfloor(\log p)^{1.5}\rfloor$ jumps to the next integer (a "depth transition" boundary), causing p and $p+2$ to straddle a change in d_0.

Example: The twin pair $(17, 19)$ is an exception: $\mathrm{RDT}(17) = 3$, $\mathrm{RDT}(19) = 2$. Here $\log(17)^{1.5}$ and $\log(19)^{1.5}$ fall on different sides of an integer threshold (roughly when $\log n \approx 2.1$). Most other twin primes, however, share the same depth.

4.2 Perfect Numbers

Even perfect numbers (of the form $n = 2^{p-1}(2^p - 1)$, with $2^p - 1$ prime) exhibit a near-linear relationship between $\mathrm{RDT}(n)$ and iterated logarithms.

Theorem 4.2 (Perfect Number Formula)**.** *If n is a perfect number of the form $2^{p-1}(2^p - 1)$, then approximately*

$$\mathrm{RDT}(n) \approx \left\lfloor \frac{\ln(\ln n)}{\ln 2} + 1 \right\rfloor.$$

In other words, $\mathrm{RDT}(n)$ *is roughly one plus the base-2 logarithm of* $\ln(\ln n)$*. Empirically, for the first few perfect numbers* $(6, 28, 496, 8128)$*, this formula gives values* $1, 2, 3, 4$ *which are off by about 1 from the actual depths* $2, 3, 4, 5$*. A refined fit is:*

$$\mathrm{RDT}(n) = \lfloor 1.45 \ln(\ln n) + 0.5 \rfloor,$$

which matches known small cases exactly.

Heuristic Derivation. For $n = 2^{p-1}(2^p - 1)$, $\ln n = (p-1)\ln 2 + \ln(2^p - 1) \approx p \ln 2$ for large p. Then $\ln(\ln n) \approx \ln(p \ln 2) \approx \ln p + \text{const}$. On the other hand, since $\mathrm{RDT}(n) \sim 2.17 \ln(\ln n)$ (Theorem 3.2), we get $\mathrm{RDT}(n) \approx 2.17 \ln p$ (plus a constant). But p itself is approximately $\log_2 n$, so $\mathrm{RDT}(n)$ scales linearly with $\log_2 p$, explaining the form of the formula above. The factor 1.45 in the refined formula is close to $2.17/\ln 2 \approx 3.13$, suggesting further adjustments for small p and floor effects.

4.3 Mersenne Primes

A related property holds for Mersenne primes $M_p = 2^p - 1$. The RDT depths of a Mersenne prime and the corresponding power of two are almost the same.

Theorem 4.3 (Mersenne Prime Depth)**.** *Let $M_p = 2^p - 1$ be a Mersenne prime. Then*

$$|\mathrm{RDT}(2^p) - \mathrm{RDT}(M_p)| \leq 1.$$

In most known cases, $\mathrm{RDT}(2^p) = \mathrm{RDT}(M_p)$*.*

For example, for $p = 7$, $2^p = 128$ and $M_p = 127$ both have RDT depth 4; for $p = 13$, 8192 and 8191 both have depth 5. The difference is at most 1 for small p (see Table 1).

p	2^p	M_p	$\mathrm{RDT}(2^p)$	$\mathrm{RDT}(M_p)$
2	4	3	2	1
3	8	7	3	2
5	32	31	4	3
7	128	127	4	4
13	8192	8191	5	5

Table 1: RDT depths for powers of 2 vs. Mersenne primes for small exponents.

Explanation. We have $\log(2^p) = p \ln 2$ and $\log(M_p) = \ln(2^p - 1) \approx p \ln 2$ (slightly smaller). Thus $(\ln(2^p))^{1.5} \approx (\ln M_p)^{1.5}$, giving $\lfloor (\ln(2^p))^{1.5} \rfloor = \lfloor (\ln M_p)^{1.5} \rfloor$ typically. Therefore the initial divisor d_0 is the same and the division chain nearly identical. At worst, M_p might require one extra step if it falls just below a threshold for some divisor. This property suggests a simple RDT-based filter for Mersenne prime candidates: require $\mathrm{RDT}(2^p)$ and $\mathrm{RDT}(2^p - 1)$ to be within 1 as a necessary (but not sufficient) condition for $2^p - 1$ to be prime.

4.4 Goldbach Partitions

For even numbers expressed as a sum of two primes ($n = p + q$), we observe that splitting into primes typically increases the RDT depth sum.

Theorem 4.4 (Goldbach RDT Inequality)**.** *If $n = p + q$ with p, q prime (a Goldbach partition of even n), then*

$$\mathrm{RDT}(p) + \mathrm{RDT}(q) \geq \mathrm{RDT}(n),$$

with strict inequality in the majority of cases.

In tested examples (e.g., $n = 100$ listed below), every prime pair partition yields a sum of depths greater than $\mathrm{RDT}(n)$. Intuitively, breaking n into prime addends usually results in a "taller" combined division tree.

Example: For $n = 100$ ($\mathrm{RDT}(100) = 3$), some partitions and depths are:

- $100 = 3 + 97$: $\mathrm{RDT}(3) + \mathrm{RDT}(97) = 1 + 3 = 4 > 3$.
- $100 = 17 + 83$: $\mathrm{RDT}(17) + \mathrm{RDT}(83) = 3 + 3 = 6 > 3$.

All Goldbach partitions of 100 give a sum strictly larger than 3.

We conjecture $\mathrm{RDT}(p) + \mathrm{RDT}(q) > \mathrm{RDT}(p + q)$ holds for almost all partitions, with equality only in trivial cases (e.g., one of the primes is 2, or $\mathrm{RDT}(n)$ is at a boundary). This suggests that "mixing" two numbers (by addition) tends to reduce logarithmic height, reflecting the fact that a composite number can be reduced more efficiently by RDT than two smaller primes separately.

4.5 Highly Composite Numbers

Highly composite numbers (HCNs), which have more divisors than any smaller number, unsurprisingly have low RDT depth relative to their size. In fact, as n grows, the ratio of the divisor count to RDT depth diverges:

Observation 4.5. *Let $\tau(n)$ be the number of divisors of n. For highly composite numbers,*

$$\tau(n)/\mathrm{RDT}(n) \to \infty \quad \textit{as } n \textit{ increases.}$$

In other words, HCNs achieve extremely large $\tau(n)$ while keeping $\mathrm{RDT}(n)$ small (growing only $\sim \log \log n$). A small table:

n	$\tau(n)$	$\mathrm{RDT}(n)$	$\tau(n)/\mathrm{RDT}(n)$
60	12	3	4.0
360	24	4	6.0
840	32	4	8.0
(larger n)	(grows)	(slow)	(unbounded)

Even though HCNs have many small prime factors (maximizing τ), RDT depth remains low because the algorithm's divisors do not directly depend on having many factors—they depend on $\log n$. This further underscores the independence from factorization (Theorem 3.4).

4.6 Fibonacci Numbers

Fibonacci numbers F_n also exhibit a simple empirical ratio: $\log_2 F_n$ divided by $\mathrm{RDT}(F_n)$ is approximately constant.

Observation 4.6.

$$\frac{\log_2 F_n}{\mathrm{RDT}(F_n)} \approx 2.5 \quad \textit{for large } n.$$

For example, $F_{25} = 75025$ has $\log_2(F_{25}) \approx 16.19$ and $\mathrm{RDT}(F_{25}) = 5$, giving $16.19/5 = 3.24$. Across $n = 7, 10, 13, 16, 19, 22, 25$, this ratio averaged 2.56 with a standard deviation of 0.47. Thus, roughly, each increment in RDT depth corresponds to multiplying the Fibonacci number's size by about $2^{2.5} \approx 5.7$. This is an empirical finding; the value 2.5 likely relates to the Fibonacci growth rate (golden ratio) in a logarithmic scale.

4.7 Depth Transition Points

Finally, we examine where RDT depth increases. Let $b(d) = \min\{n : \mathrm{RDT}(n) = d\}$ be the smallest number with depth d. These $b(d)$ mark transition points where an extra division step becomes necessary.

Observation 4.7. *The depth transition points grow super-exponentially. Approximately:*

$$b(d) \sim \exp\left(e^{d/c}\right),$$

with $c \approx 2.17$ (the same constant from Theorem 3.2). Equivalently, the jump from depth d to $d+1$ occurs around $n \approx \exp(e^{d/2.17})$.

For small depths: $b(1) = 2$ (depth 1 starts at 2), $b(2) = 4$, $b(3) = 8$, $b(4) = 64$, $b(5) \approx 1152$, which roughly aligns with the formula above (though the prediction error grows for larger d due to discrete rounding). This pattern confirms the double-exponential scale: achieving even moderately larger RDT depths requires astronomically large n. For instance, $\mathrm{RDT}(n) = 6$ first occurs at $n \approx 1.64 \times 10^4$, and by the formula $\mathrm{RDT}(n) = 10$ would occur near $n \sim \exp(e^{10/2.17}) \approx 10^{101.99}$, far beyond practical computation. Thus, RDT depth increases exceedingly slowly with n.

5 Benchmarks and Performance

We implemented the RDT algorithm in Python and measured its behavior on random large inputs. Our benchmarks confirm the $\log\log n$ growth and allow a more precise estimate of the constant c in Theorem 3.2.

From empirical data plotting $\mathrm{RDT}(n)$ vs. $\log_e \log_e n$ for n up to 10^{12}, we measure $c \approx 2.24 \pm 0.22$, slightly higher than the value 2.17 observed for smaller n. This suggests that as n grows further, the effective constant might approach a limit around 2.3. The R^2 of the fit is over 0.99, confirming an excellent log–log linear relationship.

Full benchmark results are provided in the CSV file `supp/rdt_benchmark_results.csv` in the supplemental materials.

In terms of runtime, the RDT algorithm is extremely fast. Each call involves a loop of roughly $O(\mathrm{RDT}(n))$ iterations, which grows very slowly. In our Python implementation, computing a single $\mathrm{RDT}(n)$ for a 12-digit n took on average about 0.04 milliseconds. Even 1 million random values up to 10^{12} can be processed in a few seconds on a modern laptop. This efficiency makes the RDT depth readily computable for practical ranges of n.

6 Conclusion

We introduced the Recursive Division Tree as a novel measure of integer complexity based on iterative logarithmic scaling. The RDT algorithm runs in finite time for all inputs and exhibits a double-logarithmic growth rate, independent of the integer's factorization structure. Our theoretical analysis provided bounds and an asymptotic approximation $\mathrm{RDT}(n) \sim 2.17 \log\log n$, while extensive computations refined this constant to about 2.24. Moreover, RDT uncovered surprising patterns in classical sequences—notably a 95% depth-matching property for twin primes.

Future work may explore rigorous proofs of the empirical characterizations presented, and investigate whether RDT depth (or variations of the algorithm) can shed light on unsolved problems like prime gaps or the distribution of extreme arithmetic functions. The complete implementation is available open-source for further experimentation.

A RDT Pseudocode

Algorithm 1 Compute $\mathrm{RDT}_\alpha(n)$

Require: Integer $n \geq 2$; parameter $\alpha > 0$ (default 1.5).
Ensure: Depth k such that $x_k \leq 1$ in the RDT sequence.

```
x ← n, k ← 0
while x > 1 do
    d ← max{2, ⌊(ln x)^α⌋}
    if x < d or ⌊x/d⌋ = 0 or ⌊x/d⌋ = x then
        break                                   ▷ Termination conditions
    end if
    x ← ⌊x/d⌋
    k ← k + 1
    if k > 1000 then
        break                                   ▷ Safety limit (should not trigger in practice)
    end if
end while
return k
```

Note: The above pseudocode closely mirrors our Python implementation. The division loop executes very few iterations for even astronomically large n, due to the rapid growth of the divisor d. The worst-case scenario (x halved each time) would give $k \approx \log_2 n$, but typically k is much smaller (about $2.2 \ln\ln n$ on average).

B Related Work

This paper is part of a broader research program developing recursive structure theory and its applications. The Recursive-Adic Number Field [1] extends the RDT framework by constructing a recursion-based analogue of p-adic valuation systems, defining valuations derived from recursive depth rather than prime divisibility.

Several papers develop measure-theoretic and entropy-based extensions of recursive depth. Recursive Depth Integration [3] defines depth-weighted integrals and measures, treating recursive depth as a measurable quantity. Recursive Entropy Calculus [2] introduces a depth-dependent entropy

framework with bounds on entropy growth and resonance-like effects. Recursive Geometric Entropy [4] applies these concepts to geometric subdivision, connecting hierarchy, geometry, and information measures.

The transform theory developed in RDT-Based Zeta and Laplace Transforms [5] introduces discrete zeta-like and Laplace-like operators compatible with recursive depth structures.

On the computational side, RGE-256 [6] presents an ARX-based pseudorandom number generator informed by recursive depth and structured entropy control, with empirical validation using established statistical test suites. Topological Adam [7] explores an experimental optimizer inspired by recursive structure and energy balance concepts for gradient-based optimization.

References

[1] S. Reid, "Recursive-Adic Number Field: Construction, Analysis, and Recursive Depth Transforms," Zenodo, 2025. DOI: 10.5281/zenodo.17555644

[2] S. Reid, "Recursive Entropy Calculus: Bounds and Resonance in Hierarchically Partitioned Systems," Zenodo, 2025. DOI: 10.5281/zenodo.17862831

[3] S. Reid, "Recursive Depth Integration: A Depth-Weighted Measure on the Integers," Zenodo, 2025. DOI: 10.5281/zenodo.17753502

[4] S. Reid, "Recursive Geometric Entropy: A Unified Framework for Information-Theoretic Shape Analysis," Zenodo, 2025. DOI: 10.5281/zenodo.17882310

[5] S. Reid, "RDT-Based Zeta and Laplace Transforms," Zenodo, 2025. DOI: 10.5281/zenodo.17766570

[6] S. Reid, "RGE-256: An ARX-Based Pseudorandom Number Generator with Structured Entropy and Empirical Validation," Zenodo, 2025. DOI: 10.5281/zenodo.17982804

[7] S. Reid, "Topological Adam," Zenodo, 2025. DOI: 10.5281/zenodo.17489664